

Jack Notes

Shell Scripting is simply a file that contains linux commands without a script

date

pwd

whoami

ls

createbasic.sh

date

pwd

whoami

ls

To Run $\Rightarrow$./basic.sh

Example 1.

```
#!/bin/bash
echo "Today Date is"
date
echo "current Directory"
pwd
echo "current user"
whoami
```

ls -l

chmod +x first.sh

./first.sh

Variable.

Variable is a container for store and data inside.

Name Box $\rightarrow$ JackAge Box $\rightarrow$ 23city Box $\rightarrow$ BangaloreBash fornet

name = "Jack"

age = "23"

city = "Bangalore"

Example

name = "Jack"

echo "Hello \$name"

echo "Hello welcome \$name"

echo "Goodbye \$name"

Output

Hello Jack

Hello welcome Jack

Goodbye Jack

Example 2

#!/bin/bash

name = "Jack"

Age = "23"

city = "Bangalore"

Today = \$(date)

echo "Name: \$name"

echo "Age: \$Age"

echo "city: \$city"

echo "Date: \$Today"

Example 3

#!/bin/bash

Date = \$(date)

user = \$(whoami)

directory = \$(pwd)

hostname = \$(hostname)

echo "current date: \$Date"

echo "current user: \$user"

echo "directory: \$directory"

echo "hostname: \$hostname"

Read

command accepts input from the keyboard

Example 1:

```
echo "what is your name"
read name
echo "Name: $name"
```

Output:

Jack

Example 2:

```
read -p "what is your name: " name
read -p "what is your age: " age
read -p "what is your city: " city

echo "Name: $name"
echo "Age: $age"
echo "city: $city"
```

Arithmetic Operations

+, -, *, /, %, ^

```
total=$((1))
```

This is the natural next step because you'll use numbers in many DevOps scripts (disk usage, CPU, percentage, retries, counters, etc.)

Example 1: Disk usage check

```
#!/bin/bash

read -p "Enter the Total Disk Space: " total
read -p "Enter the used space: " used

TotalFree=$((total - used))

echo "Total Free Space: $TotalFree"
```

Example 2:

```
#!/bin/bash

total=65GB
used=10GB

answer=$((total - used))

echo "Free Space: $answer"
```

If concept

-eq	Equal	10 = 10
-ne	Not Equal	10 != 20
-gt	Greater than	20 > 10
-lt	Less than	10 < 20
-ge	Greater than or Equal	18 >= 18
-le	Less than or Equal	15 <= 20

Eq. 1:

```
#!/bin/bash

containers=7

if [ $containers -ge 5 ]
then
    echo "Application is healthy"
fi
```

Eq 2

```
#!/bin/bash

read -p "Enter the value: " CPU

if [ $CPU -gt 90 ]
then
    echo "CPU is Greater High"
fi
```

```
#!/bin/bash
```

```
cpu=95
disk=85
ram=70

echo "CPU usage: $cpu%"
echo "Disk usage: $disk%"
echo "Ram usage: $ram%"

if [ $cpu -ge 90 ]
then
    echo "High CPU usage"
else
    echo "CPU usage is OK"
fi

if [ $disk -ge 80 ]
then
    echo "Disk Almost Full"
else
    echo "Disk Space Healthy"
fi

if [ $ram -ge 90 ]
then
    echo "High RAM usage"
else
    echo "RAM usage Normal"
fi
```

Loop

A (for) loop is used to perform the same task repeatedly on multiple times. In DevOps, it's commonly used to iterate through servers, Docker containers, Kubernetes Pods, files (or) deployment environments to automate repetitive tasks.

Below loop: want to Restart nginx, docker Jenkins

```
Sudo systemctl restart nginx
Docker
Jenkins
```

From Plo 1.

```
#!/bin/bash
```

(3)

```
for service in nginx docker Jenkins
do
    echo "Restarting $service..."
    sudo systemctl restart $service
done
```

Example 2 checking container's

```
#!/bin/bash
```

```
for containers in nginx Jenkins
do
    echo "checking $containers..."
    docker ps | grep $containers
done
```

Example 3 checking Disk usage on server

```
#!/bin/bash
```

```
for server in app-server db-server
do
    echo "checking Disk usage on $server"
    ssh ubuntu@$server "df -h"
done
```

Example 4

```
#!/bin/bash
```

```
echo "_____ "
echo "Deployment started"
echo "_____ "

for server in dev test staging production
do
    echo "Deploying to $server..."
done

echo
echo "Deployment completed"
```


Functions

grep

Searches for a specific word (or) pattern in the output of a file

Ex

```
docker ps | grep nginx
grep error /var/log/syslog
```

awk

Text Processing Tool, It is used to select specific column(s) or fields from command output

Ex

```
df -h
```

Output

Filesystem	Size	used	Avail	user
/dev/sda1	50G	20G	30G	40%

Now,

```
df -h | awk 'NR==2 {print $5}'
```

Output.

40%

Sed → Replace Text

Sed 's /dev/Prod/' config.txt

Out.

Prod

\$? → check whether the command is Success/Failure.

0 → Success

Non-Zero → Failure

docker

(4)

```
#!/bin/bash
```

```
docker start nginx
```

```
if [ $? -eq 0 ]
```

```
then
```

```
echo "Started Successfully"
```

```
else
```

```
echo "Failed to start"
```

```
fi
```

while loop

A while loop repeats until the condition becomes false.

Example

```
while True
```

```
do
```

```
echo "checking server.."
```

```
sleep 5 → Every Five Seconds
```

```
done
```

Cron

→ Daily Take Backup at 2.am
com
12 P.m

Task assigner.

Example 1

check whether nginx running or not
#!/bin/bash

echo " _____"

echo " Docker container Health check"

echo " _____"

echo " checking Nginx container,"

docker ps | grep nginx

if [\$? -eq 0]

then

echo "Nginx container Running"

else

echo "Nginx container Not Running"

fi

echo "check completed"

Example 2

#!/bin/bash

read -p "Enter the Environment (dev|Test|Prod) : " env

if [\$env = dev]

then
echo "Deployment to Dev..."

elif [\$env = Test]

then
echo "Deployment to Test..."

elif [\$env = Prod]

then
echo "Deployment to Prod..."

else

echo "Invalid Environment"

fi
